Project Report– Digital System Design

Smart Traffic Light Controller using Mealy FSM

BACHELOR OF ENGINEERING

Degree In

Computer Systems Engineering

Group Members:
Zain Ul Abidin Arain (CMS ID: 133-22-0001)
Soyam Kapoor (CMS ID: 133-22-0041)

Guided by:
Dr: Kashif Hussain,

Department of Computer Systems Engineering
Sukkur IBA University

November 2025

TABLE OF CONTENTS

No.	Topic / Subtopic	Page No.
1	Abstract	4
2	Problem Definition & Objectives	4
3	Block Diagram & Design Methodology	5
3.1	Block Diagram Overview	5
3.2	Design Methodology	5
3.2.1	Traffic FSM (Mealy Machine)	5
3.2.2	Timer Module	5
3.2.3	Light Decoder Module	6
3.3	Modular Design Advantages	6
4	Truth Tables & State Diagrams	7
4.1	State Transition Table	7
4.2	State Encoding	7
4.3	Next-State Logic Table	8
4.4	Next-State Logic Equations	8
4.5	Output Logic Table	8
4.6	Output Encoding	8
4.7	Output Logic Equations	8
4.8	Lights Equations	9
5	Schematic Diagram	9
6	Implementation in HDL (Verilog)	10
6.1	Traffic FSM Module	10
6.2	Timer Module	10
6.3	Light Decoder Module	11
6.4	Top-Level Module	11
6.5	Testbench	12
6.6	Simulation Results & Waveforms	13
7	Multisim Implementation	16
7.1	Timer Module Implementation	16
7.2	Counter and Synchronization Logic	16
7.3	State Register Implementation	17
7.4	Next-State & Output Logic Implementation	17
7.5	7-Segment Display for Timer Visualization	17
7.6	Results Analysis	18
8	Conclusion & Future Work	20
9	References	21
10	Appendix (Code)	22

TABLE OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Block Diagram	5
Figure 2	Schematic of Timer Module	6
Figure 3	FSM State Diagram	7
Figure 4	Schematic Diagram of Traffic Controller	9
Figure 5	FSM State Transitions (Waveforms)	13
Figure 6	Timer Counter Behaviour	13
Figure 7(a)	Main & Side Road Traffic Light Outputs – No Car Scenario	15
Figure 7(b)	Main & Side Road Traffic Light Outputs – Transition S0	14
Figure 7(c)	Main & Side Road Traffic Light Outputs – Transition S1	14
Figure 7(d)	Main & Side Road Traffic Light Outputs – Transition S2	14
Figure 7(e)	Main & Side Road Traffic Light Outputs – Transition S3	15
Figure 7(f)	Main & Side Road Traffic Light Outputs – Reset to S0	15
Figure 8(a)	FSM State Transitions in Multisim – S0 to S1	18
Figure 8(b)	FSM State Transitions in Multisim – S1 to S2	18
Figure 8(c)	FSM State Transitions in Multisim – S2 to S3	18
Figure 8(d)	FSM State Transitions in Multisim – S3 to S0	19
Figure 8(e)	FSM State Transitions in Multisim – No Car Scenario	19

1. ABSTRACT

Managing traffic efficiently at busy intersections is a major challenge in urban areas. Traditional traffic lights operate on fixed timers, which often leads to unnecessary waiting and congestion. This project presents a smart traffic light controller using a Mealy finite state machine (FSM) that adapts to real-time traffic conditions. The system uses traffic sensors to detect vehicles on both the main road (Academic Ave.) and the side road (Bravado Blvd.) and adjusts the traffic light outputs accordingly. A key feature of this design is customizable timing for each state, allowing the duration of green, yellow, and red lights to be easily modified based on traffic requirements. The controller is implemented in Verilog HDL, simulated in Vivado, and can be extended to FPGA boards for real-world testing. The modular design ensures scalability and responsiveness, improving traffic flow and reducing waiting times compared to fixed-timer systems.

2. PROBLEM DEFINITION & OBJECTIVES

Efficient traffic management is a crucial aspect of urban infrastructure, especially at busy intersections. Fixed-timer traffic lights often fail to respond to real-time traffic conditions, leading to congestion, increased travel time, and driver frustration. The goal of this project is to design a smart traffic light controller that adapts dynamically to traffic flow, using sensor inputs to determine when vehicles are waiting on either the main road (Academic Ave.) or the side road (Bravado Blvd.).

This project addresses the following key objectives:

1. **Dynamic Traffic Control:** Implement a Mealy FSM that adjusts light transitions based on both the current traffic conditions and the present state of the lights.
2. **Customizable State Timing:** Allow the duration of each light state (green, yellow, red) to be flexibly adjusted, so the system can adapt to varying traffic demands without redesigning the controller.
3. **Minimized Waiting Time:** Reduce unnecessary waiting on roads with low traffic while ensuring safety and orderly traffic flow at the intersection.
4. **HDL Implementation:** Develop the controller in Verilog HDL, verify it through simulation, and design it modularly to allow easy integration with hardware platforms like FPGAs.
5. **Simulation & Validation:** Ensure functional correctness under normal, edge, and corner-case traffic scenarios using a testbench and waveform analysis.

By achieving these objectives, the project demonstrates a scalable and practical approach to modern traffic light control systems that is more responsive than traditional fixed-timer setups.

3. Block Diagram & Design Methodology

The **Smart Traffic Light Controller** is structured into modular components, making it easy to understand, implement, and modify. The system primarily consists of three main modules: the **FSM controller**, the **timer**, and the **light decoder**. Each module has a clearly defined role in the overall operation of the traffic lights.

3.1. Block Diagram Overview

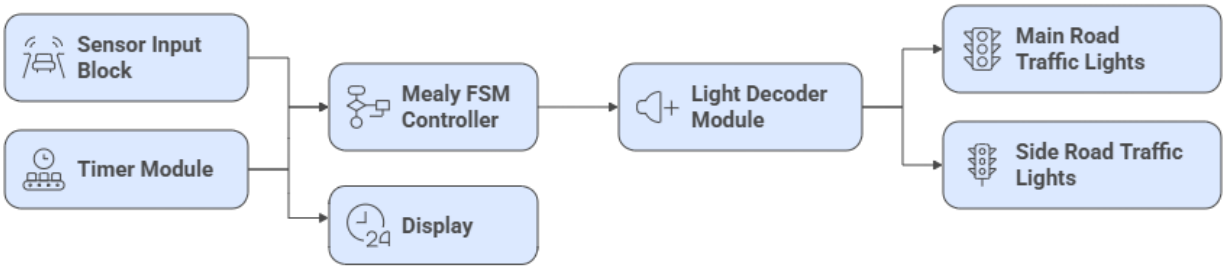


Figure 1: Block Diagram

3.2. Design Methodology

3.2.1 Traffic FSM (Mealy Machine):

- The heart of the system is a Mealy finite state machine. Unlike Moore machines, the outputs depend on both the **current state** and **traffic sensor inputs**.
- The FSM uses **two state bits**, allowing four distinct states corresponding to the main and side roads' traffic light patterns:
  - Main Green / Side Red
  - Main Yellow / Side Red
  - Main Red / Side Green
  - Main Red / Side Yellow
- **Next-state logic** evaluates the current state and sensor inputs to determine the following state.

3.2.2 Timer Module:

- Provides **customizable time intervals** for each state.
- Each state has a configurable timer value, which can be changed depending on traffic requirements without altering the FSM logic.
- When the timer expires, a signal timer_done is sent to the FSM to trigger a state transition.

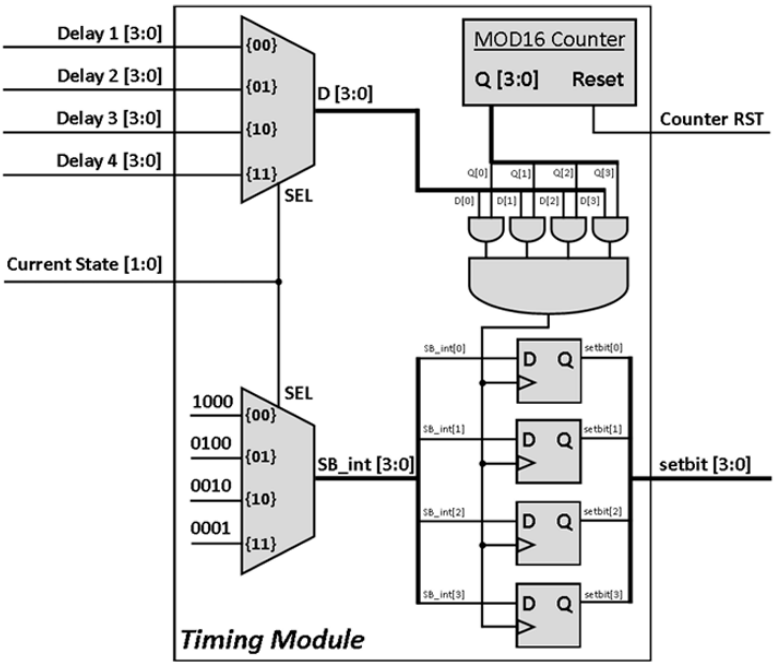


Figure 2: Schematic of Timing Module

3.2.3 Light Decoder Module:

- Converts the FSM state bits into actual **traffic light outputs**: Green, Yellow, Red for both the main road and the side road.
- Encodes the output using separate signals for each light, ensuring clear visualization on hardware (LEDs or simulation).

3.3. Modular Design Advantages

- **Scalability:** Adding new states or roads can be achieved without redesigning the entire system.
- **Flexibility:** The timer module allows adjusting green/yellow/red durations as needed.
- **Clarity:** Separating FSM, timer, and light decoder simplifies simulation, debugging, and hardware implementation.

4. Truth Tables & State Diagrams

To fully describe the behavior of our Smart Traffic Light Controller, we include both state diagrams and truth tables that map inputs, current state, and outputs to the next state. Since we are using a Mealy FSM, the outputs depend on both the current state and traffic sensor inputs.

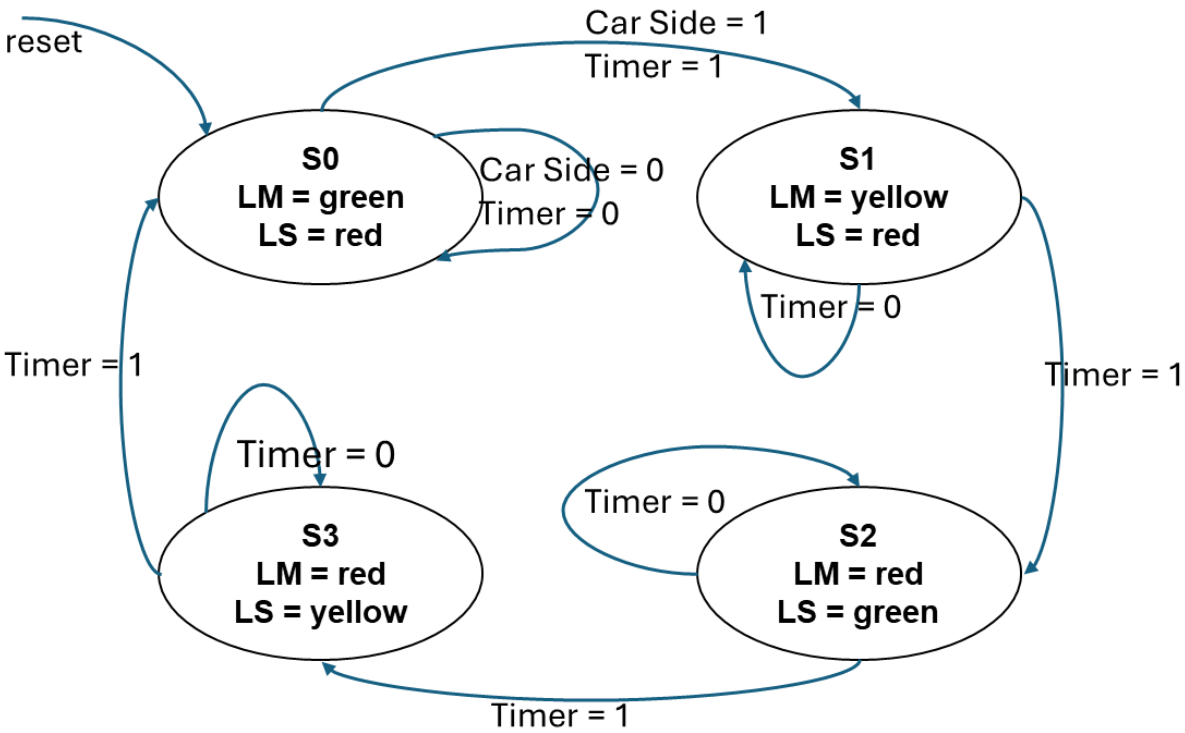


Figure 3: FSM State Diagram

4.1. State Transition table

Current State	C (Car Side)	T (Timer)	Next State
S0	0	0	S0
S0	0	1	S0
S0	1	0	S0
S0	1	1	S1
S1	X	0	S1
S1	X	1	S2
S2	X	0	S2
S2	X	1	S3
S3	X	0	S3
S3	X	1	S0

4.2. State Encoding

State Name	S_1	S_0	Description
S0	0	0	Main Green, Side Red
S1	0	1	Main Yellow, Side Red
S2	1	0	Main Red, Side Green
S3	1	1	Main Red, Side Yellow

4.3. Next-State Logic Table

Current State				Next State	
S_1	S_0	C (Car Side)	T (Timer)	S_1'	S_0'
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	X	0	0	1
0	1	X	1	1	0
1	0	X	0	1	0
1	0	X	1	1	1
1	1	X	0	1	1
1	1	X	1	0	0

4.4. Next-State Logic Equations

- $S_1' = (\bar{S_1} \cdot S_0 \cdot T) + (S_1 \cdot \bar{S_0}) + (S_1 \cdot S_0 \cdot \bar{T})$
- $S_0' = (\bar{S_1} \cdot \bar{S_0} \cdot C \cdot T) + (S_0 \cdot \bar{T}) + (S_1 \cdot \bar{S_0} \cdot T)$

4.5.Output Logic Table

Current State		Output	
S_1	S_0	LM	LS
0	0	Green	Red
0	1	Yellow	Red
1	0	Red	Green
1	1	Red	Yellow

4.6. Output Encoding

00	Green
01	Yellow
10	Red

Current State		Output			
S_1	S_0	LM_1	LM_0	LS_1	LS_0
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

4.7. Output Logic Equations

- $LM_1 = S_1$
- $LM_0 = \bar{S_1} \cdot S_0$
- $LS_1 = \bar{S_1}$
- $LS_0 = S_1 \cdot S_0$

4.8. Lights Equations:

Main Road:

- $MG = \bar{L}M_1 \cdot \bar{L}M_0$
- $MY = \bar{L}M_1 \cdot LM_0$
- $MR = LM_1 \cdot \bar{L}M_0$

Side Road:

- $SG = \bar{L}S_1 \cdot \bar{L}S_0$
- $SY = \bar{L}S_1 \cdot LS_0$
- $SR = LS_1 \cdot \bar{L}S_0$

5. Schematic Diagrams

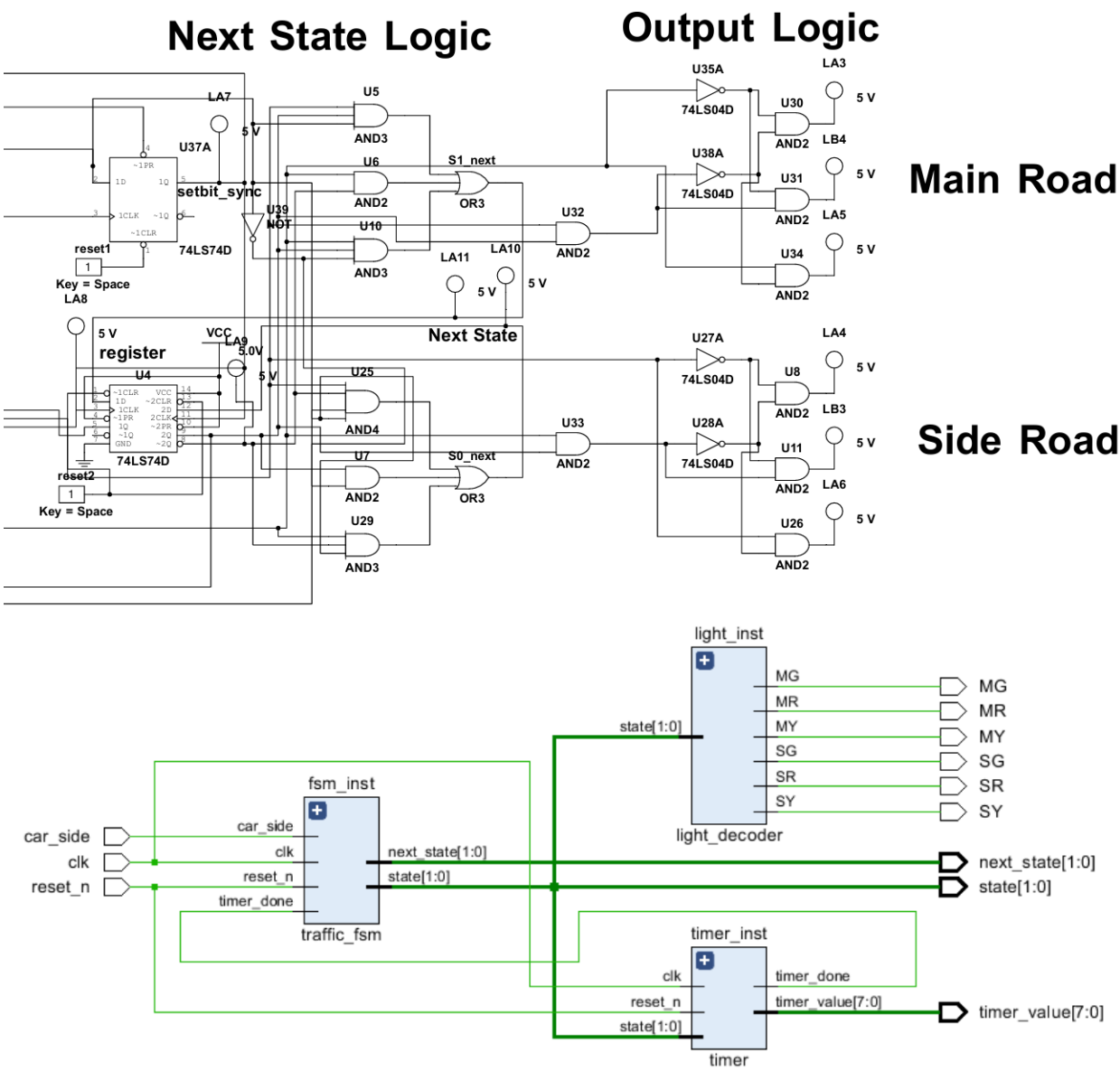


Figure 4: Schematic Diagram

6. Implementation In HDL (Verilog)

This section describes the detailed Verilog implementation of the Smart Traffic Light Controller, including module structure, signal descriptions, and simulation setup.

6.1. Traffic FSM Module

The `traffic_fsm` module implements the core Mealy FSM logic, controlling traffic light states based on the current state, side-road sensor input (`car_side`), and timer signals (`timer_done`).

Inputs:

- `clk` : Clock signal
- `reset_n` : Active-low reset
- `car_side` : Sensor input from side road
- `timer_done` : Indicates expiration of the current state timer

Outputs:

- `state` : Current FSM state
- `next_state` : Predicted next state

Implementation Details:

- Four states are defined (S0–S3) representing main and side road light combinations.
- Next-state logic uses a `case` statement based on `state`, `timer_done`, and `car_side`.
- On each positive clock edge, the FSM updates the current state; if reset is asserted, the state defaults to S0.

```
always @(*) begin
    case (state)
        S0: next_state = (timer_done && car_side) ? S1 : S0;
        S1: next_state = timer_done ? S2 : S1;
        S2: next_state = timer_done ? S3 : S2;
        S3: next_state = timer_done ? S0 : S3;
        default: next_state = S0;
    endcase
end
```

Simulation Observations:

- S0 extends when no car is present on the side road, demonstrating adaptive main-road priority.
- S1–S3 transitions occur only when the timer expires, ensuring safe and predictable traffic light changes.

6.2. Timer Module

The `timer` module provides state-dependent delays for the FSM.

Inputs:

- `clk`, `reset_n`
- `state` : Current FSM state

Outputs:

- `timer_done` : Signals completion of the current state duration
- `timer_value` : Current counter value for monitoring

Implementation Details:

- Each state has a preset target count defining its duration (`T1-T4`).
- Counter increments on every clock cycle; when the counter reaches the target, `timer_done` is asserted, and the counter resets.

```
case (state)
    2'b00: target <= T1;
    2'b01: target <= T2;
    2'b10: target <= T3;
    2'b11: target <= T4;
endcase
```

Simulation Observations:

- `timer_value` increments linearly per clock cycle.
- `timer_done` asserts precisely at the target count, confirming accurate timing for each FSM state.

6.3. Light Decoder Module

The `light_decoder` module maps FSM states to traffic light signals for both main and side roads.

Inputs:

- `state` : Current FSM state

Outputs:

- `MG, MY, MR` : Main road green, yellow, red
- `SG, SY, SR` : Side road green, yellow, red

Implementation Details:

- Each FSM state is mapped to its corresponding traffic lights via a combinational `case` statement.

```
S0: begin MG=1; SR=1; end
S1: begin MY=1; SR=1; end
S2: begin MR=1; SG=1; end
S3: begin MR=1; SY=1; end
```

Simulation Observations:

- Only one light is active per road at any time, ensuring safe intersection operation.
- Output signals switch immediately upon FSM state transitions.

6.4. Top-Level Module

The `top_traffic` module integrates the FSM, Timer, and Light Decoder modules.

Inputs:

- `clk, reset_n, car_side`

Outputs:

- Traffic lights (`MG, MY, MR, SG, SY, SR`)
- Timer value (`timer_value`)
- FSM current and next states

Implementation Details:

- Instantiates the `traffic_fsm`, `timer`, and `light_decoder` modules.
- Connects inter-module signals, such as `timer_done`, for correct timing and state updates.

```
traffic_fsm fsm_inst(
    .clk(clk),
    .reset_n(reset_n),
    .car_side(car_side),
    .timer_done(timer_done),
    .state(state),
    .next_state(next_state));
```

Simulation Observations:

- FSM and Timer work in synchronization.
- Light outputs respond correctly to state changes.
- Next-state monitoring confirms FSM logic correctness.

6.5. Testbench

The `tb` module simulates system behavior under realistic traffic scenarios.

- Generates a clock signal: `always #5 clk = ~clk`
- Applies reset at the beginning and toggles `car_side` to simulate vehicles on the side road
- Monitors FSM states, traffic light outputs, and timer values

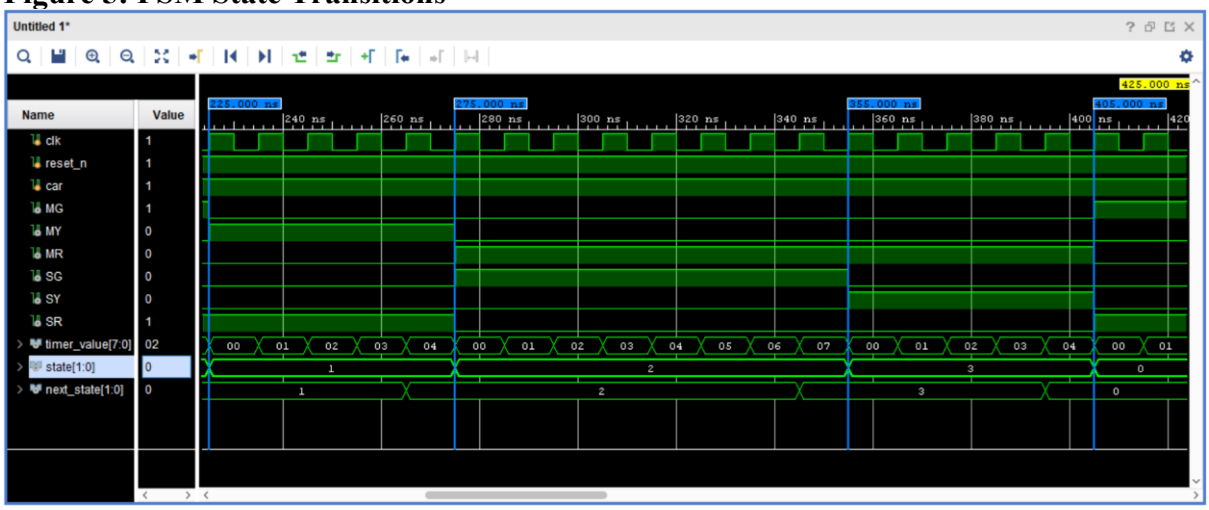
```
initial begin
    $display("TIME | Curr | Next | MG MY MR | SG SY SR | Timer |
Car");
    #20 reset_n = 1;
    #200 car = 1;
    #500 car = 0;
    #2000 $finish;
end
```

Simulation Observations:

- Main-road green extends when no car is on the side road.
- Side-road green activates only when a vehicle is detected and the FSM completes main-road phases.
- Timer durations and FSM transitions are consistent and repeatable.

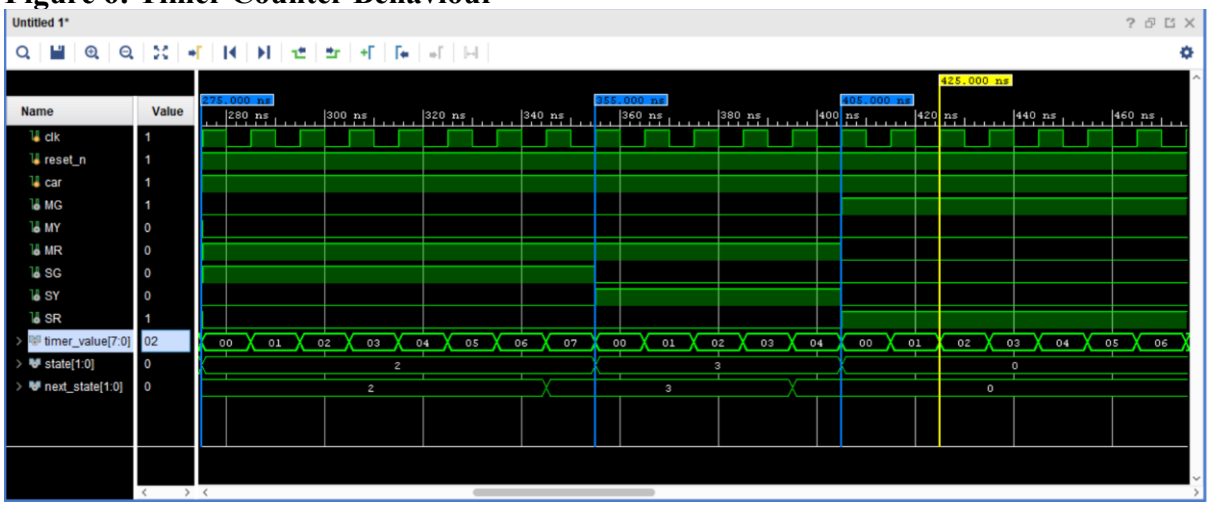
6.6. Simulation Results & Waveforms

- **Figure 5: FSM State Transitions**



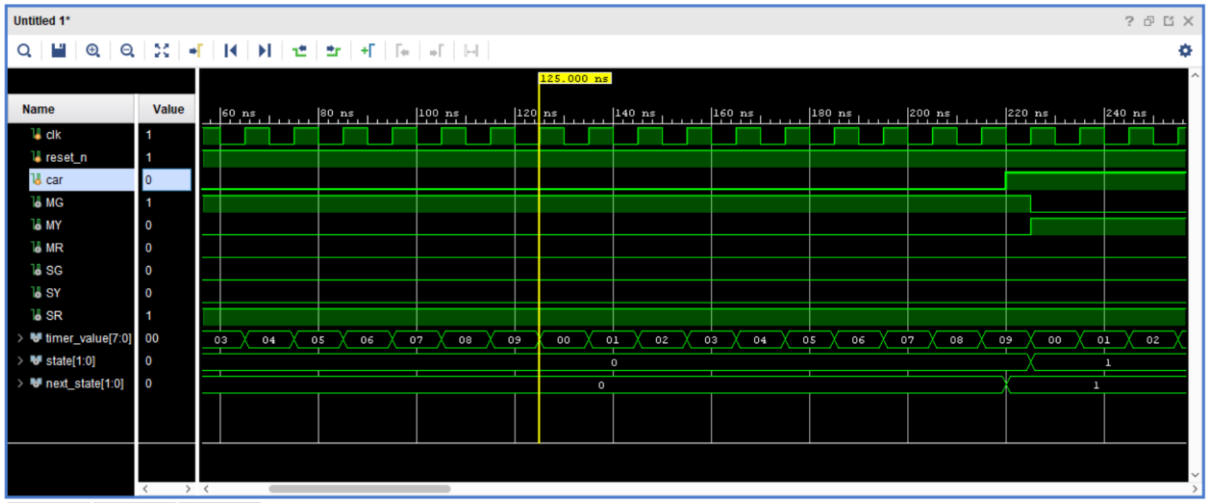
Note: blue line markers are indicating state transitions

- **Figure 6: Timer Counter Behaviour**



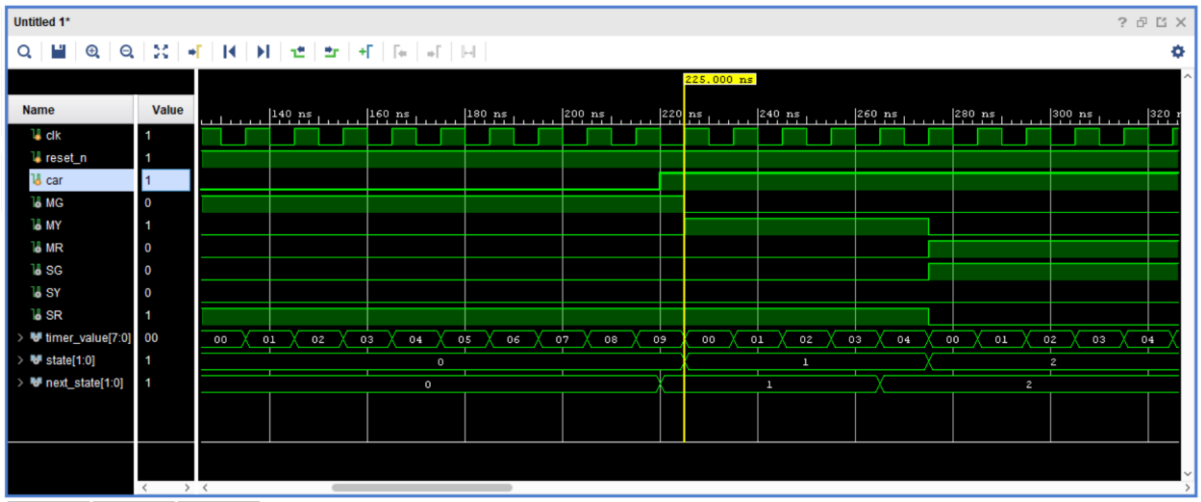
Note: highlighted waveform of timer_value indicating timers value for states i.e., 9,4,7,4 seconds for S0, S1, S2, S3 it auto resets after each state completion

- **Figure 7(a): Main & Side Road Traffic Light Outputs – No Car Scenario**



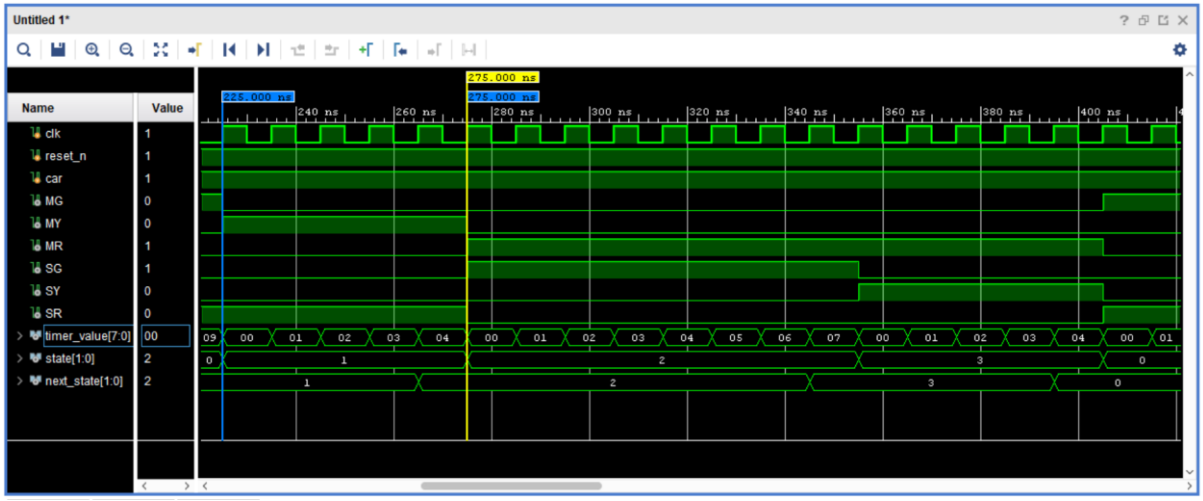
Note: we can clearly see that when there is no car the S0 remain in S0 and timer again resets for S0 again and MG is Green and SR is red.

- **Figure 7(b): Main & Side Road Traffic Light Outputs – Transition S0**



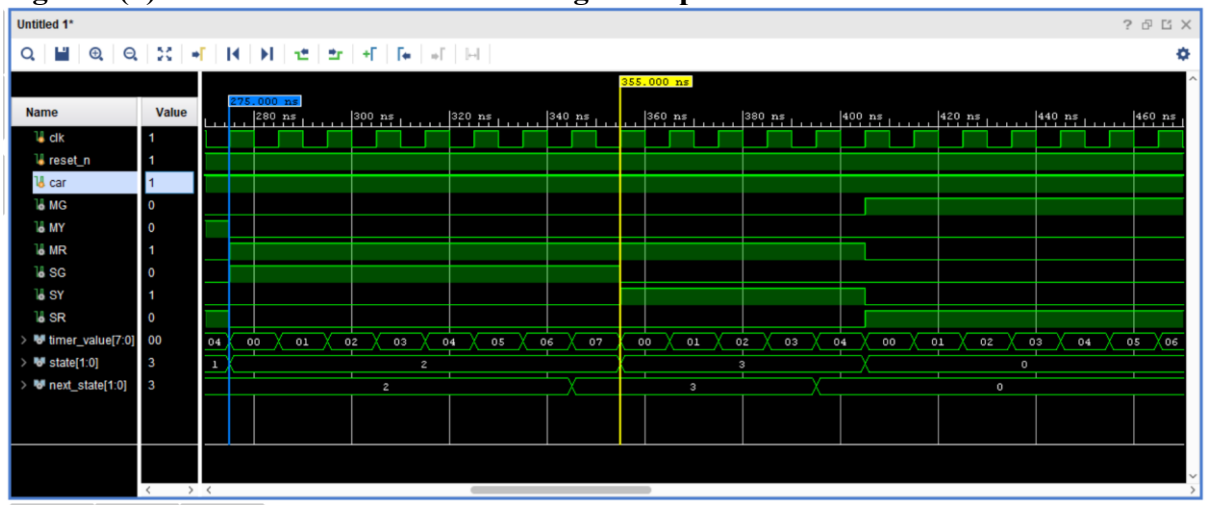
Note: we can clearly see that when car arrived the timer of S0 ended and than it transits to S1 from S0 and it will now remain in this state for 4 sec means MY & SR will be high and after that it will transit to S2

- **Figure 7(c): Main & Side Road Traffic Light Outputs – Transition S1**



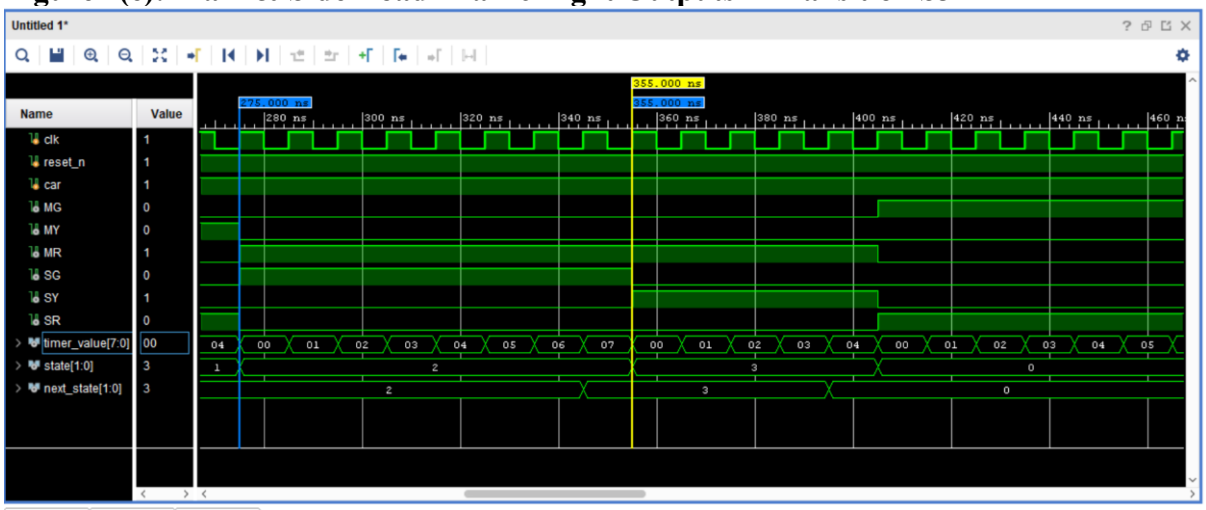
Note: we can see that the state is now transitioned in S2 successfully now it will remain in this state for 7 sec regardless of the car on the side road i.e., SG and MR are high

- **Figure 7(d): Main & Side Road Traffic Light Outputs – Transition S2**



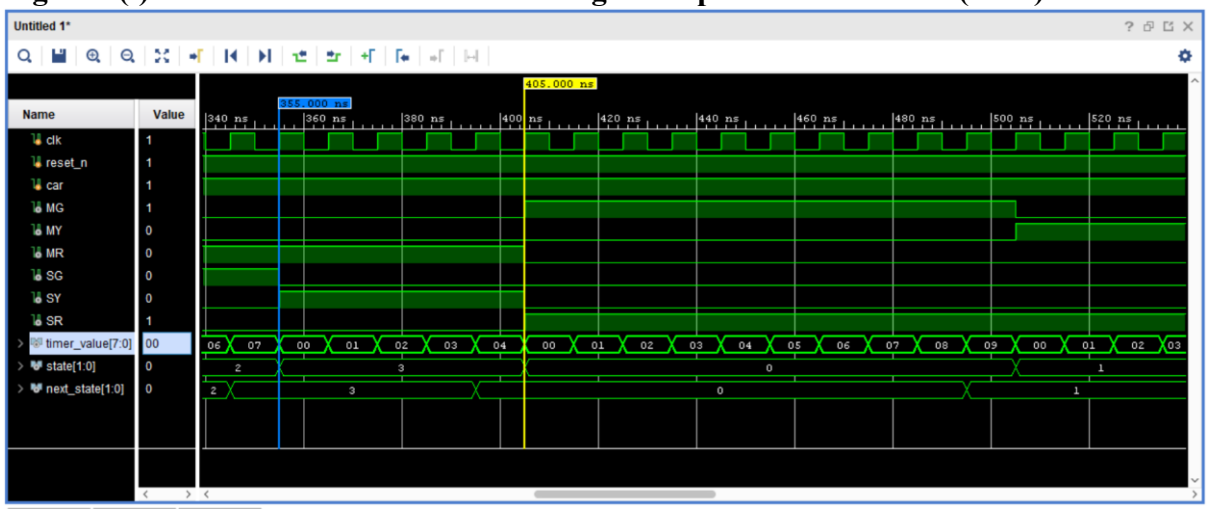
Note: we can see that after completing 7 seconds it transitioned to state S3 even thought there was a car on side road but still it transitioned and ignored the car to give the priority to main road.

- **Figure 7(e): Main & Side Road Traffic Light Outputs – Transition S3**



Note: at the final state it will remain this state for 4 sec and will move back to initial state S0 i.e., SY & MR are high

- **Figure 7(f): Main and Side Road Traffic Light Outputs Transition S0 (reset)**



Note: as we can see the states has move back to S0 now MG and SR will be high if car arrives than it will again run in same loop other wise it will remain in this state S0

7. MULTISIM IMPLEMENTATION

This section presents a complete and detailed description of how the traffic controller was implemented in Multisim using standard 74-series digital ICs. The objective of this hardware-level implementation was to reproduce the exact behavior of the FSM-based traffic controller—state transitions, timer-based control, and light-driving logic—using discrete combinational and sequential logic components.

7.1. Timer Module Implementation Using Multiplexers and Comparator

To implement a *customizable timer* for each traffic state, a modular time-selection architecture was built using two 74LS153 dual 4-to-1 multiplexers. Each 74LS153 chip contains two independent multiplexers, therefore two ICs provide a total of four 4×1 MUXes, which are sufficient for controlling all 8 bits of the timer preset.

Binary Time Presets

Each traffic state (S0, S1, S2, S3) requires a unique time duration.
These durations were encoded manually using constant binary sources in Multisim.
For example, if S1 requires 9 seconds, the binary constant 00001001 is connected as one of the inputs to the multiplexer channel.

State-Based Time Selection

The state register outputs (S1, S0) serve as the select lines (A, B) of all multiplexers.
Thus:

- When S = 00 → MUX outputs the preset time for State 1
- When S = 01 → MUX outputs the preset time for State 2
- When S = 10 → MUX outputs the preset time for State 3
- When S = 11 → MUX outputs the preset time for State 4

This guarantees that each time the FSM enters a new state, the multiplexer immediately loads the correct time constant for that state.

Comparator and Timer Done (Set Bit RAW)

The output from the multiplexers represents the target time, while an 8-bit counter produces the current elapsed time.
Both values feed a digital comparator.
When:

```
Counter Value = Selected Time Value
```

the comparator generates a high-level output referred to as **Set Bit RAW**.
This signal indicates that the timer for the current state has completed.

7.2. Counter and Synchronization Logic

Counter (74LS161D)

To measure real-time seconds, a **74LS161D 4-bit synchronous counter** was used.
Two counters were cascaded to obtain an 8-bit count range.
The counters are driven using a **1 Hz clock** generated from Multisim's clock source, ensuring that:

```
1 clock pulse = 1 second
```

The counter increments from 0 to the preset time and then must reset for the next cycle.

RAW-to-SYNC Conversion using D Flip-Flops

The comparator output (Set Bit RAW) is asynchronous with respect to the system clock, which may cause glitches.
To solve this, the raw signal is passed through a 74LS74 D flip-flop to synchronize it with the 1 Hz clock.
Output → **Set Bit SYNC**
This synchronized pulse:

- Prevents metastability
- Creates a clean, single-cycle pulse
- Safely triggers state transitions and counter reset

Automatic Counter Reset
Set Bit SYNC is passed through a NOT gate and fed back into the counters' **CLR*** pin.
Thus, when the timer completes:

- Comparator asserts Set Bit RAW
- Flip-flop synchronizes it → Set Bit SYNC
- Counter resets to 0
- FSM moves to next state

7.3. State Register Implementation

State memory was implemented using two 74LS74 D-Flip-Flops, forming a 2-bit state register.

Clocking
The clock input of the state register is driven directly by Set Bit SYNC.
This means:

- A new state loads only when the timer has completed
- No extra clocking circuitry is required
- State updates remain synchronized with the timer duration

Reset Control
A manual push-button was connected to the asynchronous reset pins of all flip-flops.
This provides the ability to force the system to initial state S1 at startup.

7.4. Next-State and Output Logic Implementation

The Boolean equations previously derived for next-state logic and output logic were implemented using:

- **74LS08 (AND gates)**
- **74LS32 (OR gates)**
- **74LS04 (NOT gates)**

These gates compute:

1. **Next State Inputs D1, D0**
   Based on the FSM transition rules (car presence, timer done).
2. **Traffic Light Outputs**
   MG, MY, MR, SG, SY, SR
   Each light is driven through an LED to visualize the controller operation.

LEDs were arranged such that:

- **Main Road:** 3 LEDs (Red, Yellow, Green)
- **Side Road:** 3 LEDs (Red, Yellow, Green)

This fully reproduces the complete functioning of a traffic signal head.

7.5. 7-Segment Display for Timer Visualization

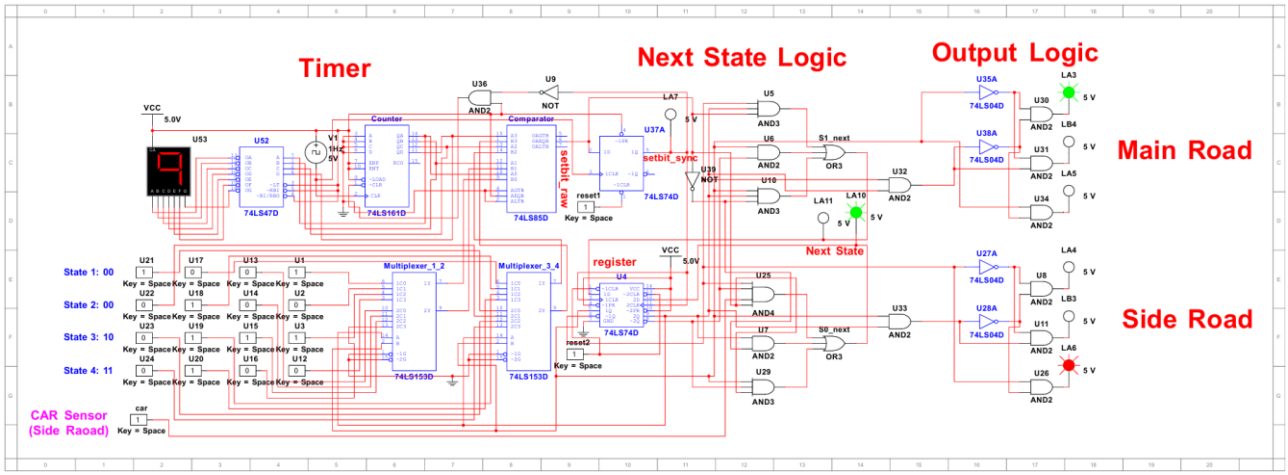
To visualize the countdown timer, a dual-digit 7-segment display was connected through a BCD-to-7SEG converter.
Connections:

- The counter output feeds the BCD converter.
- Converter drives the 7-segment display.
- The display updates once per second due to the 1 Hz clock.

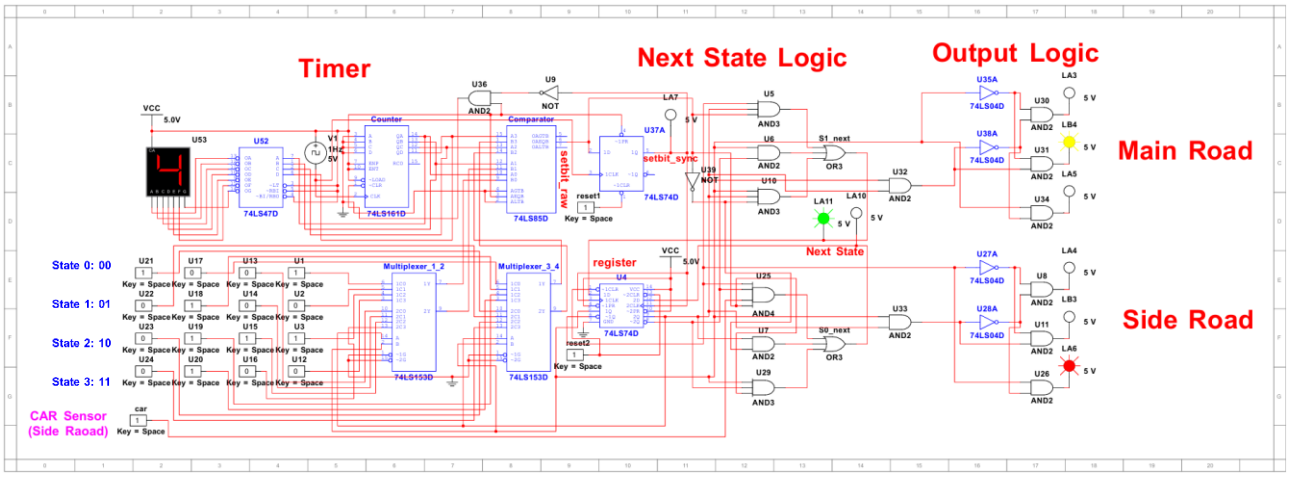
This provides real-time feedback of the remaining time in each state.

7.6. Results Analysis

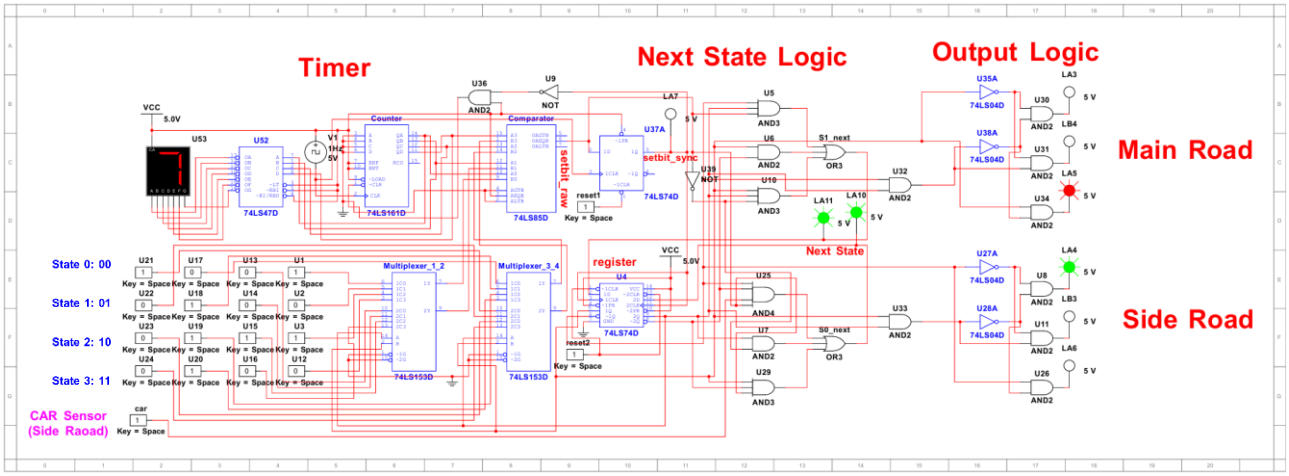
- **Figure 8(a): FSM State Transitions in Multisim – S0 to S1**
  State: S0 (LM = green & LS= red) and when car = 1 and 9 sec done, it will transit to S1



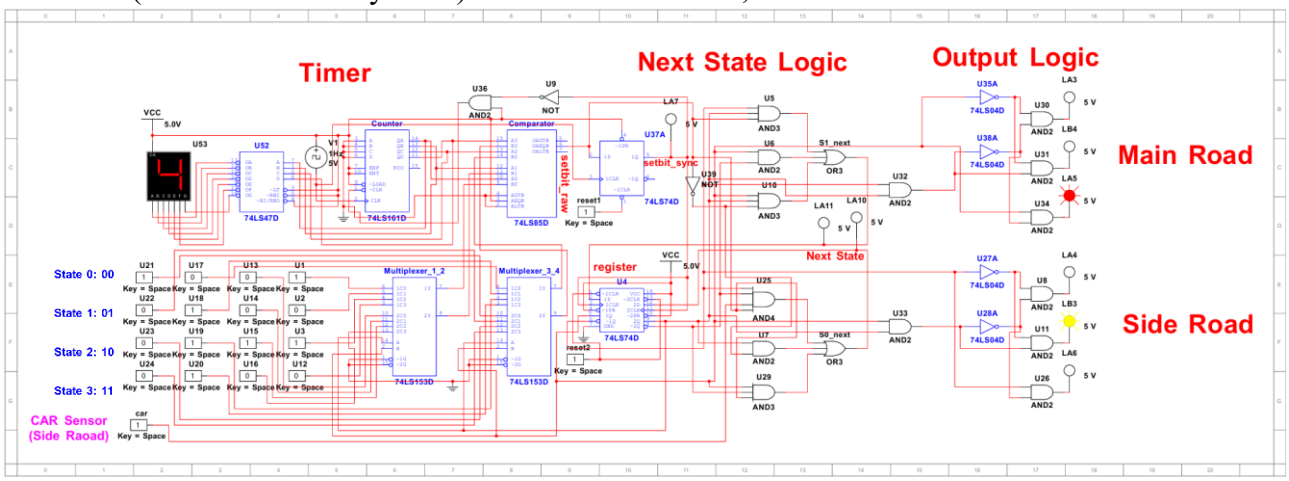
- **Figure 8(b): FSM State Transitions in Multisim – S1 to S2**
  State: S1 (LM = yellow & LS= red) and when 4 sec done, it will transit to S2



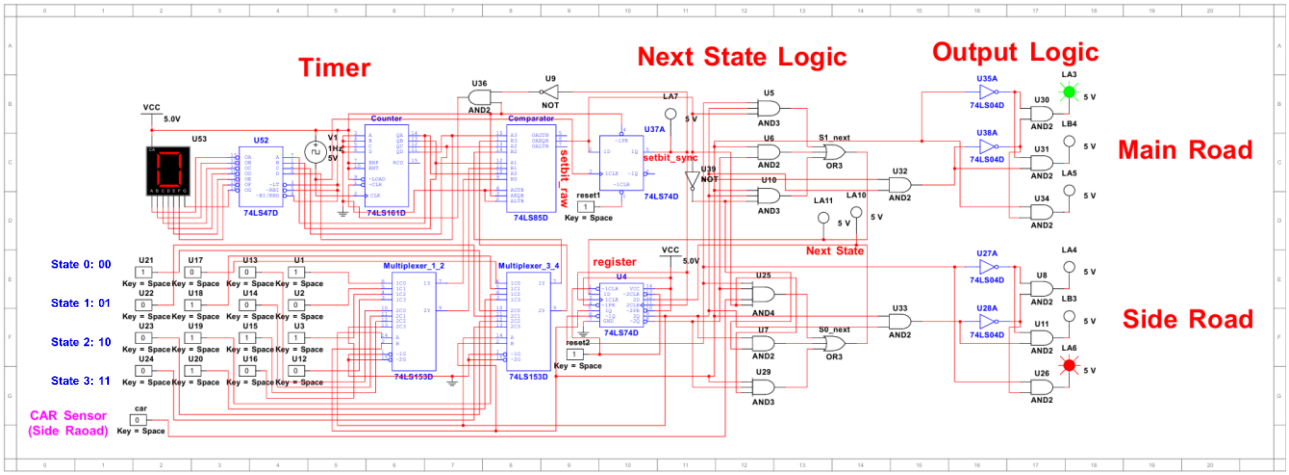
- **Figure 8(c): FSM State Transitions in Multisim – S2 to S3**
  State: S2 (LM = red & LS= green) and when 7 sec done, it will transit to S3



- **Figure 8(d): FSM State Transitions in Multisim – S3 to S0**
  State: S3 (LM = red & LS= yellow) and when 4 sec done, it will transit to S0



- **Figure 8(e): FSM State Transitions in Multisim – No Car Scenario**
  If there is no car (car = 0) the it will remain in S0 and the timer will gain restart for the S0 until the car arrives



The Multisim simulation results strongly validate the design. Every module like counter, comparator, multiplexer, flip-flop, state register, and output logic integrates seamlessly into a coherent synchronous system. The behaviour shows that the traffic controller is robust and ready for real hardware implementation.

8. CONCLUSION & FUTURE WORK

This project successfully implemented a complete traffic light controller system using two different platforms: Verilog HDL and Multisim with discrete logic ICs. Both implementations were based on the same four-state FSM and identical timing logic, and both produced the same behavior during testing.

The Verilog implementation provided fast simulation, easy debugging, and a clear functional view of the system, while the Multisim version demonstrated the real hardware behavior of counters, multiplexers, comparators, flip-flops, and logic gates. Despite their different environments, both systems showed consistent state transitions, accurate timing, correct light outputs, and proper response to the side-road car sensor.

The results confirm that the FSM design, timer mechanism, and output logic are correct and robust. This work demonstrates a full digital design cycle rom theoretical equations to HDL implementation and finally to hardware-level realization proving that the system can be reliably deployed on FPGA or in real hardware.

The current traffic light controller can be further enhanced to improve efficiency and real-world applicability. One key improvement is adaptive timing, where green, yellow, and red durations are dynamically adjusted based on real-time traffic density, reducing unnecessary waiting. Integration of pedestrian signals with push-buttons would improve safety at intersections.

The system could also be extended to coordinate multiple intersections, enabling smoother traffic flow across urban areas. Implementing the controller on an FPGA or microcontroller would allow practical deployment and testing with actual sensors and lights. Adding fault tolerance mechanisms would increase reliability in case of timer or sensor failures, and an enhanced user interface such as an LCD or GUI could provide real-time monitoring of traffic light states and timer values. Collectively, these enhancements would make the system smarter, safer, and more adaptable to real-world traffic conditions.

9. REFERENCES

1. M. Morris Mano and Michael D. Ciletti, *Digital Design*, 5th Edition, Pearson Education, 2013.
   (Used for FSM design methodology, state diagrams, and logic implementation.)
2. Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2019.
   (Referenced for Verilog coding structure, synchronous logic, and FSM modelling.)
3. Xilinx Inc., *Vivado Design Suite User Guide: Synthesis (UG901)*, 2022.
   (Used for understanding Verilog simulation and synthesis behavior.)
4. Xilinx Inc., *Vivado Design Suite User Guide: Getting Started (UG910)*, 2022.
   (Referenced for running simulations, waveform viewing, and project setup.)
5. Texas Instruments, *74LS153 Dual 4-Line to 1-Line Multiplexer Datasheet*, TI, 2017.
   (Used for implementing time-select MUX in Multisim.)
6. Texas Instruments, *74LS161 Synchronous 4-Bit Counter Datasheet*, TI, 2017.
   (Referenced for timer counter configuration.)
7. Texas Instruments, *74LS74 Dual D-Type Flip-Flop Datasheet*, TI, 2015.
   (Used for synchronizing comparator output and building state register.)
8. National Instruments (NI), *Multisim User Guide*, Version 14.2, 2020.
   (Referenced for simulation setup, IC usage, and circuit design.)
9. P. Ashok and K. Karthik, "Design of Traffic Light Controller Using Verilog HDL," *International Journal of Engineering Research & Technology (IJERT)*, vol. 6, 2017.
   (General reference related to traffic controller implementation.)
10. IEEE Standard 1364-2005, *Verilog Hardware Description Language*.
    (Referenced for syntax and behavioral modelling rules.)

10. APPENDIX

Code:

```
//================ Traffic FSM ===================
`timescale 1ns/1ps
module traffic_fsm (
   input  wire      clk,
   input  wire      reset_n,
   input  wire      car_side,
   input  wire      timer_done,
   output reg  [1:0] state,
   output reg  [1:0] next_state
);
// State encoding
localparam S0 = 2'b00;   // Main GREEN , Side RED
localparam S1 = 2'b01;   // Main YELLOW, Side RED
localparam S2 = 2'b10;   // Main RED   , Side GREEN
localparam S3 = 2'b11;   // Main RED   , Side YELLOW
always @(*) begin
   case (state)
      S0: next_state = (timer_done && car_side) ? S1 : S0;
      S1: next_state = timer_done ? S2 : S1;
      S2: next_state = timer_done ? S3 : S2;
      S3: next_state = timer_done ? S0 : S3;
      default: next_state = S0;
   endcase
end
always @(posedge clk or negedge reset_n) begin
   if (!reset_n)
      state <= S0;
   else
      state <= next_state;
end
endmodule

// ================== Timer ==================
`timescale 1ns/1ps
module timer (
   input  wire      clk,
   input  wire      reset_n,
   input  wire [1:0] state,
   output reg       timer_done,
   output reg [7:0]  timer_value
);
localparam T1 = 9;
localparam T2 = 4;
localparam T3 = 7;
localparam T4 = 4;
reg [7:0] counter = 0;
reg [7:0] target  = T1;
always @(posedge clk or negedge reset_n) begin
   if (!reset_n) begin
      counter     <= 0;
      timer_done  <= 0;
      target      <= T1;
      timer_value <= 0;
   end
   else begin
      case (state)
         2'b00: target <= T1;
         2'b01: target <= T2;
         2'b10: target <= T3;
         2'b11: target <= T4;
      endcase
      if (counter >= target) begin
         timer_done <= 1;
         counter    <= 0;
      end
      else begin
         timer_done <= 0;
         counter    <= counter + 1;
      end
      timer_value <= counter;
   end
end
endmodule

//================ Light Decoder =================
`timescale 1ns/1ps
module light_decoder (
   input  wire [1:0] state,
   output reg MG, MY, MR,
   output reg SG, SY, SR
);
localparam S0 = 2'b00;
localparam S1 = 2'b01;
localparam S2 = 2'b10;
localparam S3 = 2'b11;
always @(*) begin
   MG = 0; MY = 0; MR = 0;
   SG = 0; SY = 0; SR = 0;
   case (state)
      S0: begin MG=1; SR=1; end
      S1: begin MY=1; SR=1; end
      S2: begin MR=1; SG=1; end
      S3: begin MR=1; SY=1; end
   endcase
end
endmodule

// ================ Top Module ==================
`timescale 1ns/1ps
module top_traffic (
   input  wire clk,
   input  wire reset_n,
   input  wire car_side,
   output wire MG, MY, MR,
   output wire SG, SY, SR,
   output wire [7:0] timer_value,
   output wire [1:0] state,
   output wire [1:0] next_state
);
wire timer_done;
traffic_fsm fsm_inst(
   .clk(clk),
   .reset_n(reset_n),
   .car_side(car_side),
   .timer_done(timer_done),
   .state(state),
   .next_state(next_state)
);
timer timer_inst(
   .clk(clk),
   .reset_n(reset_n),
   .state(state),
   .timer_done(timer_done),
   .timer_value(timer_value)
);
light_decoder light_inst(
   .state(state),
   .MG(MG), .MY(MY), .MR(MR),
   .SG(SG), .SY(SY), .SR(SR)
);
endmodule

// ================ Testbench ==================
`timescale 1ns/1ps
module tb;
reg clk = 0;
reg reset_n = 0;
reg car = 0;
wire MG, MY, MR;
wire SG, SY, SR;
wire [7:0] timer_value;
wire [1:0] state;
wire [1:0] next_state;
top_traffic dut(
   .clk(clk),
   .reset_n(reset_n),
   .car_side(car),
   .MG(MG), .MY(MY), .MR(MR),
   .SG(SG), .SY(SY), .SR(SR),
   .timer_value(timer_value),
   .state(state),
   .next_state(next_state)
);
always #5 clk = ~clk;
initial begin
   $display("TIME | Curr | Next | MG MY MR | SG SY SR | Timer | Car");
   #20 reset_n = 1;
   #200 car = 1;
   #500 car = 0;
   #2000 $finish;
end
always @(posedge clk) begin
   $display("%4t |  %b    %b  | %b  %b  %b | %b  %b  %b | %d | %b",
      $time,
      state, next_state,
      MG, MY, MR,
      SG, SY, SR,
      timer_value,
      car
   );
end
endmodule
```